

Robot Autonomy - Homework 1

Prof. Oliver Kroemer

TAs: Shua Halle, Srishti Gupta

This homework will focus on the topics of kinematics and control from the lectures. You will explore force and impedance control on a simple scene in the MuJoCo (Multi-Joint Dynamics with Contact) simulator for the first part, and then implement forward and inverse kinematics for the Franka arm. To set up MuJoCo for the homeworks instructions have been provided in the next section. An additional introductory pdf has been provided to get you started with MuJoCo once you have set up the simulator. The homeworks have been tested on Ubuntu. If you require a virtual machine or help with the installation, please let me know. You are encouraged to work together and discuss questions with your peers, but please write your own code for the implementations.

1 - MuJoCo Setup

To set up MuJoCo,

1. Navigate to the home directory
2. Create a python virtual environment. To do this, run
`python3 -m venv ./<your_env_name>.`
Replace `your_env_name` with whatever you'd like.
3. Activate this environment by running the activate script within the bin folder of the venv path you provided, i.e. run
`source ./<your_env_name>/bin/activate`
4. Within this environment, run `which python` and ensure that the python path is pointing to a location in the venv path as a sanity check.
5. Install dependencies using the environments pip package by running
`python -m pip install mujoco`
`python -m pip install quaternion`
`python -m pip install matplotlib`
6. You can now use the packages from the Python venv to run the homework files. Make sure to activate the venv when you run the files.

Note: You may use any Python environment manager that you prefer. `venv` is an example, not a requirement.

IMPORTANT: Ensure there are no whitespaces in your folder names as this can cause problems while running the MuJoCo executable, especially when working inside Python environments.

MacOS: To run Python scripts that use MuJoCo, use the `mjpython` binary to run them, i.e., replace the usual `python script.py` with `mjpython script.py`

2 - PID Control

In this section, you will implement PID controllers in a simple MuJoCo scene to familiarize yourself with the simulation environment and understand the basics of PID control. You will find two scripts located in the Control folder: `PandaArmControl_Part1.py` and `PandaArmControl_Part2.py`. The primary difference between these files lies in the world being loaded. In Part 1, the environment contains the Franka arm and a static whiteboard, while in Part 2, the Franka arm interacts with a moving whiteboard.

2.1 - Control with a Fixed Wall

In Part 1, you are expected to implement an impedance controller and a force controller such that the arm generates 15 N of force against the whiteboard as shown in Figure 1.

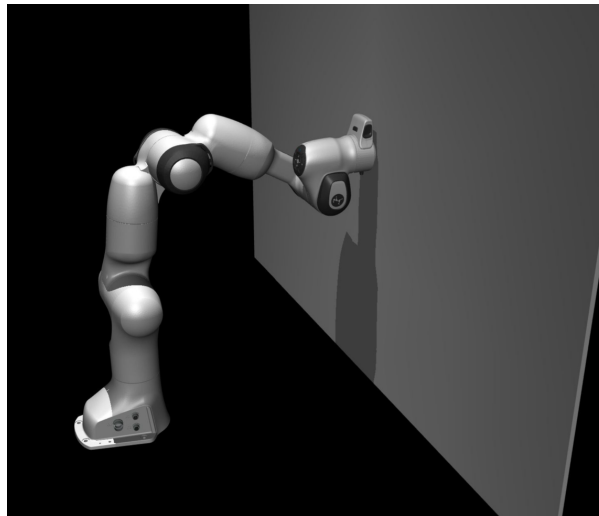


Figure 1: Franka arm applying force against a static whiteboard in MuJoCo.

To simplify the process, the robot is initialized with a predefined set of joint positions. This initial state is stored in the MJCF model file and is loaded each time the simulator is launched. You are encouraged to review the script to understand how the control pipeline is structured. Briefly:

1. The main function initializes the `mjModel` and `mjData` data structures, sets the relevant keyframe for the homework, and registers the control callback function.
2. The control callbacks are defined above the main function. You are encouraged to experiment with gravity compensation and position control to better understand how the robot interacts with the environment. For more details on interacting with the simulator, refer to the Intro to MuJoCo PDF provided with this handout.

2.2 - Control with a Moving Wall

In this section, you will use the same controllers implemented in Part 1, but introduce disturbances in the form of a sinusoidally moving whiteboard. Copy your controller implementations from the previous section into `PandaArmControl_Part2.py` and run the file. Observe and analyze how the robot behaves under this disturbance.

2.3 - Plotting the Effects

Both scripts will output CSV files containing time and force values recorded over a 10-second simulation window. Ensure that the simulator is run for at least 10 seconds for each controller (Impedance and Force control) for both the fixed and the moving whiteboard wall.

You may use any plotting tool of your choice to visualize force versus time. Be sure that each plot includes a title, axis labels, and a legend.

Submission

For the control section, submit a single PDF page containing the following items:

- A screenshot of the force plots for the static whiteboard case. Plot the outputs of the force controller and impedance controller superimposed on the same figure. Ensure the plot includes a title, axis labels, and a legend clearly identifying each controller.
- A screenshot of the force plots for the oscillating whiteboard case. Plot the outputs of the force controller and impedance controller superimposed on the same figure, again including a title, axis labels, and a legend.
- A screenshot of the robot when the whiteboard is near its maximum amplitude (i.e., closest to the robot).
- A brief (2–3 sentence) explanation describing the differences in behavior between the force controller and the impedance controller.
- A brief (2–3 sentence) overview of your observation and analysis on how the robot behaves under this disturbance.

3 - Kinematics

3.1 - Forward Kinematics

In this section, you will implement the forward kinematics (FK) for the Franka arm and compute the Jacobian. Please use Python for your implementation. Starter code has been provided in `Franka.py` and `RobotUtil.py` in the Kinematics folder. You may use the helper functions in `RobotUtil.py` or implement your own as needed.

Your task is to implement the computation of the forward kinematics and Jacobian in the `ForwardKin()` method of the `FrankaArm` class in `Franka.py`. Your FK method should output the transformation matrices for each joint based on the input joint angles, as well as the Jacobian matrix.

To test your implementation, run your FK method for the following joint configurations and compute the resulting end-effector poses. Submit your computed end-effector poses for each configuration listed below. (Note: these joint configurations are also defined as a list named `joint_targets` in `MainTest.py`.)

Targets:

- $\mathbf{q}_1 = [0^\circ \ 0^\circ \ 0^\circ \ 0^\circ \ 0^\circ \ 0^\circ \ 0^\circ]$
- $\mathbf{q}_2 = [0^\circ \ 0^\circ \ -45^\circ \ -15^\circ \ 20^\circ \ 15^\circ \ -75^\circ]$
- $\mathbf{q}_3 = [0^\circ \ 0^\circ \ 30^\circ \ -60^\circ \ -65^\circ \ 45^\circ \ 0^\circ]$

Expected output from FK for $\mathbf{q}_1$:

$$\mathbf{t} = \begin{bmatrix} 0.088 \\ 0 \\ 0.926 \end{bmatrix}$$

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

3.2 - Inverse Kinematics

In this section, you will implement the inverse kinematics (IK) for the Franka arm using the damped least-squares method. For the weighting matrices, use the following definitions for diagonal matrices C and W .

$$C = \text{diag}([1000000, 1000000, 1000000, 1000, 1000, 1000])$$

$$W = \text{diag}([1, 1, 100, 100, 1, 1, 100])$$

Please use Python for your implementation. Write your code in the IK method `IterInvKin()` of the `FrankaArm` class in `Franka.py`.

Test your IK implementation: Run your IK method using the starting joint configuration and target end-effector pose defined in `Main.py` (referred to as `qInit` and `HGoal` in the code, respectively), and compute the joint configuration that achieves the desired end-effector pose. Note: the `qinit` given below is in radians.

$$\mathbf{q}_{\text{init}} = \begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \\ -2.11 \\ 0.0 \\ 3.65 \\ -0.785 \end{bmatrix}, \quad \mathbf{t}_g = \begin{bmatrix} 0.6 \\ 0 \\ 0.5 \end{bmatrix}, \quad \mathbf{R}_g = \begin{bmatrix} 0.0 & 0.0 & 1.0 \\ 0.0 & 1.0 & 0.0 \\ -1.0 & 0.0 & 0.0 \end{bmatrix}.$$

Submission

For the kinematics section, submit a single PDF containing the following items:

- The three end-effector poses (in matrix form) corresponding to `q1`, `q2`, and `q3`.
- The final joint angles that achieve the target end-effector pose ($\mathbf{R}_g, \mathbf{t}_g$).
- Use the position controller callback to move the robot to the joint configuration returned by your inverse kinematics solver, and include an image of the robot at this configuration.